

Experiment # 11

Implementation of Priority Scheduling Algorithm

Objective

In this lab we will learn the implementation of Priority Scheduling Algorithm.

Software Tools

- ◆ Ubuntu 16.04
- ◆ GCC Compiler

Theory

Priority scheduling is one of the most common scheduling algorithms in batch systems. Each process is assigned a priority. Process with the highest priority is to be executed first and so on. Processes with the same priority are executed on first come first served basis. Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Characteristics of Priority Scheduling

- A CPU algorithm that schedules processes based on priority.
- It used in Operating systems for performing batch processes.
- If two jobs having the same priority are READY, it works on a FIRST COME, FIRST SERVED basis.
- In priority scheduling, a number is assigned to each process that indicates its priority level.
- Lower the number, higher is the priority.
- In this type of scheduling algorithm, if a newer process arrives, that is having a higher priority than the currently running process, then the currently running process is preempted.

Difference between Primitive and non-primitive:

Primitive:

1. In Preemptive Scheduling, the CPU is allocated to the processes for a specific time period.
2. In Preemptive Scheduling, tasks are switched based on priority

Non-Primitive:

1. In non-preemptive scheduling CPU is allocated to the process until it terminates.
2. In non-Preemptive Scheduling, tasks, no switching takes place.

Algorithm

1. Input the processes along with their burst time (bt).

2. Get the priority for each process and value.
3. Start with the higher priority process from its initial position let other processes to be in queue i.e. sort processes according to their priorities.
4. If more than 1 process has same burst time, then FCFS algorithm will be followed.
5. Find waiting time (wt) for all processes.
6. As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. $wt[0] = 0$.
7. Find **waiting time** for all other processes i.e., for process i,
 $wt[i] = bt[i-1] + wt[i-1]$.
8. Find **turnaround time** = waiting_time + burst_time for all processes.
9. Find **average waiting time** = total_waiting_time / no_of_processes.
10. Similarly, find **average turnaround time** = total_turn_around_time / no_of_processes.
11. Compute Average Waiting Time of processes.
12. Compute Average Turnaround Time of processes.
13. Display the Gantt chart.

Output:

```

Enter number of processes:3
Enter burst time and priorities of processes
Burst Time for Process P1:5
Enter Priority of process P1:3
Burst Time for Process P2:7
Enter Priority of process P2:1
Burst Time for Process P3:9
Enter Priority of process P3:2
p2: burst time:7      priority:1
p3: burst time:9      priority:2
p1: burst time:5      priority:3

Process Details

```

Process No.	burst time	priority	waiting time	turnaround time
P2	7	1	0	7
P3	9	2	7	16
P1	5	3	16	21

```

Gantt Chart is as follows

```

Process	Start Time	End Time
P2	0	7
P3	7	16
P1	16	21

```

Average waiting time is:7.66667
Average turnaround time is:7
Finish Time is:21

```

LAB TASK

1. Implement priority scheduling algorithm for preemptive processes.

Process ID	Arrival Time	Burst Time	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

2. Implement priority scheduling algorithm for non-preemptive processes.

Process ID	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.